Property Management Suite

Configuration & End-to-End User Guide

Odoo 18 Enterprise — 10-Module Suite

Single Property / Multi-Tenant Operations Edition

Prepared for: **Techverse Online Limited** (odooerp.co.ke)
Author: Geronimo · peterkamere@gmail.com
June 2026 · Version 18.0.1.0.0

Contents

1. Suite Overview & Architecture
2. Prerequisites & Installation Order
3. Security Groups & Access Rights
4. Initial Configuration (Step by Step)
5. Core Concepts & Data Model
6. End-to-End Walkthrough: One Property, Several Tenants
7. Rent Billing Cycle: Schedules, Invoices, Payments, Penalties
8. Maintenance Operations
9. Landlord Settlement & Owner Payouts
10. Agent Commissions
11. CRM: From Lead to Signed Tenancy
12. Property Sale & Purchase with Installments
13. Reporting Catalogue
14. Module-by-Module Reference
15. Integration with Other Odoo Modules
16. Scheduled Jobs & Automatic Emails
17. Suggested Enhancements (Roadmap)
18. Appendix A — Odoo 18 Migration Notes
19. Appendix B — Maintenance & Upgrade Procedure

1. Suite Overview & Architecture

The suite manages the full lifecycle of rental real estate: registering properties and units, onboarding tenants and landlords, signing tenancy contracts, scheduling and invoicing rent, charging late-payment penalties, running maintenance, paying property owners, rewarding agents, and selling/purchasing properties on installment plans — all on standard Odoo accounting.

Module map

Module	Purpose	Depends on
`property_management_ee`	**Core.** Properties, tenants, tenancy contracts, rent schedules, invoicing, CRM bridge, reports, security	account, account_asset (Enterprise), analytic, sale_crm
`property_landlord_management_ee`	Landlords (owners), landlord tenancies, owner rent payout invoices	core
`property_maintenance_ee`	Property maintenance requests, stage emails, re-invoicing to tenant	core + maintenance
`property_recurring_maintenance_ee`	Recurring maintenance charges attached to a tenancy	property_maintenance_ee
`property_penalty_ee`	Late-payment penalties on rent schedules	core
`property_booking_ee`	Bulk unit generation (towers/floors/units), booking & availability, split/merge	core
`property_commission_ee`	Agent commissions on rent / buy / sell, commission invoicing & reports	core
`property_rent_report_ee`	Rent voucher / payment-status report	core
`property_sale_purchase_ee`	Property sale & purchase with installment plans from payment terms	core
`property_ee`	Umbrella bundle — installs the nine modules above in one click	all of the above

> **Design key:** a **Property is an Account Asset** (`account.asset` , flag `is_property`) so every property is simultaneously a depreciable fixed asset in Accounting. A **Tenancy contract is an Analytic Account** (`account.analytic.account`) so every rupee/shilling of

> rent, penalty, maintenance and owner payout is traceable per contract through analytic accounting.

2. Prerequisites & Installation Order

Prerequisites

- Odoo **18 Enterprise** (the suite extends the Enterprise `account_asset` module).
- Apps already installed: **Accounting**, **Assets** (account_asset), **CRM**, **Sales**, **Maintenance**.
- A configured **Chart of Accounts** and at least one company with currency set (e.g. KES).
- Outgoing mail server configured (Settings → Technical → Email) — the suite sends rent reminders and maintenance notifications.

Installation order

1. `property_management_ee` (core — everything depends on it)
2. `property_maintenance_ee` (needed before recurring maintenance)
3. `property_landlord_management_ee`, `property_penalty_ee`, `property_booking_ee`, `property_commission_ee`, `property_rent_report_ee`, `property_sale_purchase_ee` — any order
4. `property_recurring_maintenance_ee` (after maintenance)
5. *Optional:* `property_ee` if you prefer one-click installation of the whole suite.

After installing or updating module files on the server always: upload to `/opt/odoo18/custom_addons/` → restart the Odoo service → Apps → module → **Upgrade** (views and data live in the database).

3. Security Groups & Access Rights

The core module creates a dedicated application category with four user roles. Assign them in Settings → Users & Companies → Users.

Group	Who	Capabilities
Property Manager (`group_property_manager`)	Your staff	Full create/read/write on properties, tenancies, rent schedules, configuration menus, reports
Property User / Tenant (`group_property_user`)	Tenants	Sees only properties they currently occupy and their own tenancy/maintenance records (record rule on `current_tenant_id`)
Property Owner (`group_property_owner`)	Landlords	Sees only properties they manage/own (record rule on `property_manager`)
Property Agent (`group_property_agent`)	Brokers	Sees properties they handle; access to commission records

> **Automatic user provisioning:** creating a **Tenant** or **Landlord** record automatically creates the linked `res.partner` and can provision a portal/internal user with the right group, so tenants can log in and see their contract. Deleting a tenant with active tenancies is blocked.

> Give accounting staff the normal **Accounting / Billing** rights as well — rent invoices, payments and asset depreciation post into standard journals.

4. Initial Configuration (Step by Step)

4.1 Accounting prerequisites

> **Step 1 — Income account for rent.** Accounting → Configuration → Chart of Accounts: create/identify an **Income** account, e.g. *700100 Rental Income*.
> **Step 2 — Maintenance income/expense account**, e.g. *700200 Maintenance Recharges*.
> **Step 3 — Asset accounts** for depreciation: *Fixed Asset (Buildings)*, *Accumulated Depreciation*, *Depreciation Expense*, and a **Miscellaneous/Assets journal**.
> **Step 4 — Payment terms** used for property sale installments (e.g. *12 monthly installments*): Accounting → Configuration → Payment Terms.

4.2 Analytic plan

The suite auto-creates an analytic plan named **"Property Management"** the first time a tenancy is created; every tenancy contract is an analytic account on this plan. Nothing to do manually — just be aware it appears under Accounting → Configuration → Analytic Plans.

4.3 Property configuration menus

> **Property Management → Configuration:**
> • **Property Types** — e.g. Apartment, Office, Shop, Godown, Stand-alone House.
> • **Rent Types** — billing frequency presets (Monthly, Quarterly, Yearly…).
> • **Property Labels** (booking module) — unit numbering schemes (ABCD / 101 / 1001 / 10001 prefixes).
> • **Maintenance Teams** (Maintenance app) — e.g. Plumbing, Electrical, General; add team members.

4.4 Email & scheduled actions

> • Verify outgoing mail server works (send a test email).
> • Settings → Technical → Scheduled Actions: confirm **"Scheduled Rent"** (daily) and **"Property States Changed"** (every 10 minutes) are active.
> • Settings → Technical → Email Templates: review *Tenancy: Mail to Tenant* (rent reminder) and the four *Maintenance In Progress/Done* templates; adjust wording/branding.

4.5 Penalty policy (property_penalty_ee)

> On each tenancy contract you can set the penalty parameters (percentage/amount per overdue period). Agree the house policy first — e.g. *5% of rent after the 5th of the month* — and apply it consistently when creating tenancies.

4.6 Commission policy (property_commission_ee)

> On the **Agent** partner record set the commission percentages for *rent*, *buy*, and *sell* transactions. Commissions are then computed per tenancy/deal and invoiced from the commission screen.

5. Core Concepts & Data Model

Business object	Technical model	What it holds
Property / Unit	`account.asset` (`is_property`)	Address & geo-location, type, furnishing, bedrooms/bathrooms/parking, GFA (ft²/m²), unit price, ground rent, photos, floor plans, rooms, utilities, nearby places, safety certificates, insurance, attachments, **parent/child hierarchy** for buildings→units, depreciation board, manager, current tenant, ROI & occupancy KPIs
Tenancy contract	`account.analytic.account`	Tenant, property, rent amount & frequency, deposit (received/returned), start/end dates, rent schedule lines, penalty settings, contract document, state (Draft → Open → Close/Renew/Cancel)
Rent schedule line	`tenancy.rent.schedule`	One expected rent payment: due date, amount, penalty amount, invoice link, paid flag; buttons to invoice & open the invoice
Tenant	`tenant.partner` (delegates to `res.partner`)	Contact data + occupation, documents, tenancies; auto user provisioning
Landlord / Owner	`landlord.partner` (delegates to `res.partner`)	Owner data, owned properties, landlord tenancies (payout contracts)
Maintenance request	`maintenance.request` (extended)	Property, tenant, team, stage, invoice for re-charged works
Unit factory	`property.created` (delegates to asset)	Tower/floor/unit batch generation with label prefixes; booking status
Sale/Purchase installments	fields + `sale.cost` / `purchase.cost` lines on the asset	Sale price, customer, payment term → generated installment lines

Property states

Draft (being prepared) → **Open/Normal** (available & operating) → **Book** (reserved, booking module) → **Close / Sold / Cancel** (end of cycle). The 10-minute cron keeps states consistent with bookings and tenancy changes.

6. End-to-End Walkthrough: One Property, Several Tenants

This is your exact scenario: **one building with several rentable units, each unit occupied by its own tenant**. The suite models this with the property **parent/child hierarchy**: the building is the parent property; each unit (flat, office, shop, room) is a **child property**; each child has its own tenancy contract.

> **Rule of thumb:** one tenancy contract = one rentable unit = one tenant at a time. Never attach several simultaneous tenants to a single property record — instead create one child unit per tenant. The History/Sub-Properties tab on the building gives the consolidated view.

6.1 Register the building (parent property)

> Property Management → Property Details → **New**.
> 1. **Property Name:** e.g. *Kamere Towers*. Fill **Address**, City, Country; click **Get Location** to verify the map URL (latitude/longitude).
> 2. Set **Property Type** (e.g. Apartment Block), **Property Manager** (your staff user), No. of Towers/Floors, GFA.
> 3. Accounting tab: set **Income Account** (Rental Income), **Maintenance Account**, **Depreciation** & **Expense** accounts and **Journal**; set **Original Value** (building cost) and depreciation method/duration if you depreciate it.
> 4. Add **photos, floor plans, safety certificates** (with expiry dates — they feed the expiry report), and **insurance** records.
> 5. Click **Open** to move it from Draft to operating state. Save.

6.2 Create the units (child properties)

Option A — manual (a few units): on the building form, tab **Sub Properties**, add lines; or create new properties and set their **Parent Property** to the building. Set per-unit type (1-bed, 2-bed…), furnishing, unit price (monthly rent guide), floor.

Option B — bulk (many units), booking module: Property Management → Property Booking → **Create Property**: choose the parent/tower, the floor count, units per floor and a **label prefix** (101-style, 1001-style or lettered). The wizard generates all child unit records in one pass. Units can later be **split** or **merged** with the dedicated wizards.

> Each unit inherits the accounting configuration pattern of the parent — still verify the income account on a sample unit. The building's **Sub Properties** tab shows per-unit state (Available / Booked / Occupied) and the **Available** counter.

6.3 Onboard tenants

Property Management → Tenant Details → **New**.
1. Name, phone, email, ID/passport scan as attachment, occupation, emergency contact.
2. Saving creates the linked contact and (if enabled) a **portal user** in the *Property User* group so the tenant can log in and see their tenancy & invoices.
3. Repeat per tenant — e.g. 8 tenants for 8 occupied units of Kamere Towers.

6.4 Sign a tenancy contract per unit

Property Management → Tenancy Details → **New** (one per unit+tenant):
1. **Tenant:** pick the tenant. **Property:** pick the *unit* (not the building!).
2. **Rent** amount and **Rent Type** (Monthly), **Start Date** / **Expiration Date** (e.g. 12 months).
3. **Deposit:** enter amount; after collecting it press **Deposit Received** (creates the payment entry). On exit, **Deposit Return** reverses it.
4. Penalty tab (if installed): set the late-fee policy for this contract.
5. Attach the signed lease PDF on the contract document field / chatter.
6. Press **Create Rent Schedule** — the system generates one `tenancy.rent.schedule` line per period for the whole duration. Press **Start** to open the contract. The unit's state flips to occupied and **Current Tenant** is set on the unit.

The contract is an analytic account: every invoice line it generates carries its **analytic distribution**, so P&L per unit and per building rolls up automatically in Accounting analytics.

6.5 Month after month

See Section 7 for the billing cycle. In daily use the building manager lives in three screens: **Tenancy Details** (contracts & schedules), **Maintenance** (requests per unit), and the **Property form's stat buttons** (Tenancy count, journal Entries).

6.6 Tenant turnover

1. Tenancy → **Close** (or **Cancel Tenancy** for early break). Outstanding schedules can be invoiced or dropped; **Deposit Return** settles the deposit (net of damages — post a separate invoice for damages first).
2. The unit returns to **Available**; it appears again in booking availability.
3. New tenant → new tenancy on the same unit (history of all previous tenancies stays on the unit).
4. For a renewal of the same tenant use **Renew** (renew wizard) — it extends/regenerates schedules without re-creating the contract.

7. Rent Billing Cycle

7.1 The schedule

Each tenancy holds its **Rent Schedule** lines: due date, amount, penalty, invoice status. The daily cron **Scheduled Rent** emails tenants whose rent is due (template *Tenancy: Mail to Tenant*).

7.2 Invoicing

> On each schedule line press the **gear (Create Invoice)** button — it creates a customer invoice on the tenant with the contract's income account and the tenancy's analytic distribution; the line then shows the **arrow (Open Invoice)** button. Validate & send the invoice from Accounting as usual. Bulk emailing is available via the **Mailing** action on selected schedule lines.

7.3 Payments & reconciliation

Register payments on the invoice (Accounting). The payment form carries **Tenancy** and **Property** context fields and shows the **Amount Due**. Once paid, the schedule line is flagged paid; the rent voucher report (`property_rent_report_ee`) prints the payment status per contract.

7.4 Penalties (property_penalty_ee)

Overdue schedule lines accrue a **penalty amount** per the contract policy; the penalty is added on top of the rent when the line is invoiced (or invoiced separately), and totals appear on the tenancy.

7.5 Deposits

Deposit lifecycle is tracked on the contract: **Receive** posts the deposit in, **Open Received** shows the entry, **Return** pays it back on contract end. Deposits are payments, not income — they sit on the partner balance until returned or offset.

8. Maintenance Operations

> 1. Tenant (portal or phone) or manager raises a request: Maintenance → **New**: pick the **Property (unit)** — the tenant is filled from the unit's current tenant; pick the **Maintenance Team** and priority; describe the issue; **Open Google Map** jumps to the unit location.
> 2. Move the request through stages (kanban). On **In Progress** and **Done**, automatic emails go to the tenant and the property manager (4 templates).
> 3. If works are re-charged to the tenant, press **Create Invoice** on the done request — it bills the tenant using the property's **Maintenance Account** with the tenancy analytics.

4. **Recurring maintenance** (e.g. monthly service charge, garbage, security): on the tenancy, add recurring maintenance lines — they are added into the rent schedule/invoicing cycle automatically.

Use separate teams per trade and the standard Maintenance kanban/calendar for planning; everything stays per-unit traceable through the property link.

9. Landlord Settlement & Owner Payouts

When you manage buildings on behalf of owners, install `property_landlord_management_ee`:

1. **Landlord Details** → New: owner contact; link the properties they own (*Property Owner* on the property).
2. Create a **landlord tenancy** (resident type *Landlord Partner*): the **Landlord Rent** is the payout (e.g. collected rent minus your management fee), with its own schedule.
3. Each due line: **Create Landlord Invoice** generates a **vendor bill** to the owner (purchase journal); pay it like any supplier bill. **Deposit Pay / Deposit Received** handle owner-side deposits.
4. The owner (Property Owner group) can log in and see only their properties and payout history.

Per-building profitability = rent income (tenant tenancies) − owner payouts (landlord tenancies) − maintenance, all on the same analytic plan — read it in Accounting → Reporting → Analytic Items grouped by plan/account.

10. Agent Commissions

1. Mark the broker's contact as an **Agent** and set commission % for rent / buy / sell.
2. On a tenancy closed by an agent press **Create Commission** — a commission record computes the amount from the rent and the agent's rate.
3. From the commission record, **Create Invoice** raises the vendor bill to the agent; **Open Invoice** tracks it. Commission analysis prints from the commission report menu.

11. CRM: From Lead to Signed Tenancy

1. Leads captured in CRM get a property section: **Is Rent / Is Buy / Is Demand**, desired **Property**, and a **Demand Info** tab (min/max bedrooms, bathrooms, price range, furnishing, facing, type) for requirement matching.
2. Qualify the lead in the pipeline as usual.
3. On the opportunity: **Create Quotation** (sale order for buy deals) or **Create Contract** — the wizard asks for start/end dates and converts the lead into a **tenant + tenancy contract** automatically.
4. Continue from Section 6.4 (schedule, start, invoice).

12. Property Sale & Purchase with Installments

Selling a property: on the property form (sale tab): set **Customer**, **Sale Date**, **Sale Price** and a **Payment Term**; press **Generate Payment Entries** — the installment plan (dates, amount, remaining, %) is computed from the payment term. **Create Invoice** bills each installment; the property moves to **Sold** at the end of cycle.
Purchasing: symmetric fields (purchase price, recurring rule, return period) generate the purchase installment lines for properties you acquire.

Use realistic payment terms (e.g. “30% advance, balance over 12 months”) — the plan derives 1:1 from the term lines.

13. Reporting Catalogue

Report (menu: Reporting)	Tells you
Tenancy Detail / Tenancy by Tenant	All contracts with rent & deposit, filtered by property or tenant, date-ranged
Contract Expiry	Tenancies expiring in a window — renewals pipeline
Safety Certificate Expiry / Document Expiry	Compliance documents about to lapse per property
Income / Expenditure	Per-property income vs expense totals with currency formatting
Property per Location	Portfolio split by city/region
Rent Voucher (rent report module)	Per-contract payment status voucher
Commission Report	Agent commission statements
Property Hierarchy	Building → units listing with states

Plus everything native: Accounting analytic reports per tenancy/building, Maintenance KPIs, CRM pipeline analysis.

14. Module-by-Module Reference

property_management_ee (core)

- **Menus:** Property Details, Tenancy Details, Tenant Details, Rent Schedules, Reporting, Configuration.
- **Models added/extended:** `account.asset` (+60 property fields, hierarchy, state machine), `account.analytic.account` (tenancy), `tenancy.rent.schedule`, `tenant.partner`, `property.type`, `property.rent.type`, rooms/phases/photos/floorplans/utilities/nearby/certificates/insurance/attachments, `account.move(.line)` + `account.payment(.register)` (property & tenancy context), `crm.lead` (demand), `sale.order(.line)` (property on lines), `res.partner/res.users/res.company` (roles & defaults).
- **Wizards:** renew tenancy, CRM make sale/contract, mass mail to tenants, 7 report wizards.
- **Crons:** Scheduled Rent (daily), Property States Changed (10 min). **Mail template:** rent reminder.

property_landlord_management_ee

- `landlord.partner`, landlord tenancies on the analytic model (`resident_type = landlord_partner`), landlord rent & vendor-bill generation, owner views/menus, owner record rules.

property_maintenance_ee

- Extends `maintenance.request` / `maintenance.stage`: property & tenant link, stage-routed emails (tenant + manager × in-progress + done), tenant re-invoicing via the property's maintenance account, Google-Maps shortcut, property stat button for maintenance count.

property_recurring_maintenance_ee

- Recurring maintenance lines on the tenancy; charges merge into the rent schedule cycle.

property_penalty_ee

- Penalty configuration on the tenancy; `penalty_amount` accrual on overdue `tenancy.rent.schedule` lines; penalty shown and invoiced with rent.

property_booking_ee

- `property.created` (delegated asset) unit factory: towers/floors/units with label prefixes; book/available states with kanban; split & merge wizards; `property.label` numbering config.

property_commission_ee

- Agent flags & rates on partner; `commission.invoice` records from tenancies; vendor-bill creation; commission QWeb report; partner search filters for agents.

property_rent_report_ee

- Rent voucher QWeb report with computed payment status per schedule line.

property_sale_purchase_ee

- Sale fields (customer, date, price, payment term) + **Generate Payment Entries** (installments from `payment.term._compute_terms()`), purchase installments (price, recurring rule, return period), invoice creation per installment.

property_ee

- Umbrella manifest; no code — installs the full suite.

15. Integration with Other Odoo Modules

Odoo app	How the suite plugs in
Accounting	Rent/maintenance/penalty/commission documents are real invoices & vendor bills; payments carry tenancy/property context; everything is analytic-tagged per contract on the auto-created “Property Management” analytic plan
Assets (Enterprise)	Properties *are* assets: original value, depreciation board, journal entries, sell/dispose flow available from Accounting → Assets for the same records
CRM	Demand capture on leads; one-click conversion to quotation or tenancy contract
Sales	Property attached to order lines; buy-deal quotations from CRM
Maintenance	Teams, stages, kanban reused; property/tenant context and invoicing added
Email / Discuss	Rent reminders, maintenance notifications, chatter & followers on every key record
Portal	Tenants/owners get portal logins scoped by record rules to their own data and invoices
Documents (your instance)	Lease PDFs, certificates and IDs attached on records can be centralised into Documents workspaces with an automation rule
Contacts	Tenants, landlords and agents are first-class partners — dedupe, merge and enrich as usual

16. Scheduled Jobs & Automatic Emails

Automation	Frequency	Action
Scheduled Rent	Daily	Emails tenants whose rent schedule is due (rent reminder template)
Property States Changed	Every 10 min	Reconciles property states with bookings/tenancies
Maintenance stage mails	On stage change	In-progress & done notifications to tenant and manager

17. Suggested Enhancements (Roadmap)

Quick wins

- **M-Pesa / payment-provider integration** — let tenants pay rent invoices online from the portal (Odoo payment providers; a Daraja/M-Pesa connector is the natural fit for the Kenyan market) with automatic reconciliation against the rent schedule.
- **Automatic monthly invoicing** — extend the daily cron to also *create and email* the invoice when a schedule line falls due (today it reminds; invoicing is one click per line).
- **Lease-renewal activity automation** — auto-schedule an activity to the property manager 60/30 days before contract expiry (data is already in the Contract Expiry report).
- **WhatsApp/SMS notifications** — rent due & receipt acknowledgements via Odoo's WhatsApp connector.
- **Tighten the unused booking flow** if you don't pre-sell units, or expose it as a reservation+deposit flow if you do.

Medium term

- **Utility sub-metering & billing** — water/electricity readings per unit with consumption lines added to the monthly invoice (new model + invoice-merge into the schedule cycle).
- **Service-charge pools** — building-level costs (security, cleaning, lifts) apportioned to units by GFA or fixed share, reconciled annually.
- **Owner statements** — one PDF per landlord per month: collected rent, fees retained, maintenance deducted, net payout (composition of existing data; a QWeb report away).
- **Occupancy & arrears dashboard** — spreadsheet/Studio dashboard: occupancy %, arrears aging, renewal funnel, maintenance SLA.
- **Shared tenancies** — allow co-tenants (many2many) on one contract with split invoicing percentages.

Structural / housekeeping

- **Restore asset workflow buttons** (Sell/Dispose, Pause, Modify Depreciation) on the property form using your server's exact v18 Enterprise method names (removed during migration because the v15 names no longer exist).
- **Test suite refresh** — the unit tests were partially modernised; full green tests under `--test-enable` would protect future upgrades.
- **Replace the remaining legacy UI bits** — `terp-*` filter icons (ignored by v18), the `deprec_lines_reversed` widget column if it renders oddly.
- **Localisation** — Swahili translations (`i18n/` .po files) for tenant-facing emails and the portal.

18. Appendix A — Odoo 18 Migration Notes

The suite was migrated from Odoo 15 to Odoo 18 Enterprise (June 2026). Highlights of what changed (full detail in `CHANGES_LOG.md` in this folder):

- All view syntax modernised: `attrs/states` → Python expressions, `tree` → `list`, legacy chatter → `<chatter/>`, removed `field_parent`, `js_class`, `src_model`, empty modifier expressions.
- Cron records: removed `numbercall` / `doall`.
- Enterprise asset alignment: removed `asset_type`, `prorata` → `prorata_computation_type`, `account_analytic_id` / `analytic_tag_ids` → `analytic_distribution`, `reversal_move_id` → `reversal_move_ids`, removed dead buttons whose methods no longer exist.
- Accounting APIs: `payment_term.compute()` → `_compute_terms()`; payments use `payment_method_line_id`; analytic amounts via `analytic_distribution`; tenancy History tab now lists analytic items (move-line O2M no longer possible in v18).
- Python: `formatLang(env,…)` fix, `ustr` → `str`, selection `ondelete` policy fix, `furnished` field restored on CRM leads, view_mode `tree` → `list`.

19. Appendix B — Maintenance & Upgrade Procedure

Deploying a code change:
1. Copy the changed module folder(s) to `/opt/odoo18/custom_addons/` on the server.
2. `sudo systemctl restart odoo18` (always needed for Python; cheap insurance for XML).
3. Apps → module → **Upgrade** (reloads views/data stored in DB), or from shell: `odoo -d <db> -u <module> --stop-after-init`.
4. Hard-refresh the browser (Ctrl-Shift-R) to bust the assets cache.

Before any upgrade: take a database backup (Settings → Database Manager or `pg_dump`). Property data = money data.

Support workflow that worked well: paste the full RPC traceback; the file and line in the *ParseError/view error context* identifies the offending record precisely.